

Using uv with PyTorch

The PyTorch ecosystem is a popular choice for deep learning research and development. You can use uv to manage PyTorch projects and PyTorch dependencies across different Python versions and environments, even controlling for the choice of accelerator (e.g., CPU-only vs. CUDA).

!!! note

Some of the features outlined in this guide require uv version 0.5.3 or later. We recommend upgrading prior to configuring PyTorch.

Installing PyTorch

From a packaging perspective, PyTorch has a few uncommon characteristics:

- Many PyTorch wheels are hosted on a dedicated index, rather than the Python Package Index (PyPI). As such, installing PyTorch often requires configuring a project to use the PyTorch index.
- PyTorch produces distinct builds for each accelerator (e.g., CPU-only, CUDA). Since there's no standardized mechanism for specifying these accelerators when publishing or installing, PyTorch encodes them in the local version specifier. As such, PyTorch versions will often look like 2.11.0+cpu, 2.11.0+cu130, etc.
- Builds for different accelerators are published to different indexes. For example, the +cpu builds are published on <https://download.pytorch.org/whl/cpu>, while the +cu130 builds are published on <https://download.pytorch.org/whl/cu130>.

As such, the necessary packaging configuration will vary depending on both the platforms you need to support and the accelerators you want to enable.

To start, consider the following (default) configuration, which would be generated by running `uv init --python 3.14` followed by `uv add torch torchvision`.

In this case, PyTorch would be installed from PyPI, which hosts CPU-only wheels for Windows and macOS, and GPU-accelerated wheels on Linux (targeting CUDA 13.0, as of PyTorch 2.11.0):

```
[project]
name = "project"
version = "0.1.0"
requires-python = ">=3.14"
dependencies = [
    "torch>=2.11.0",
    "torchvision>=0.26.0",
]
```

This is a valid configuration for projects that want to use CPU builds on Windows and macOS, and CUDA-enabled builds on Linux. However, if you need to support different platforms or accelerators, you'll need to configure the project accordingly.

Using a PyTorch index

In some cases, you may want to use a specific PyTorch variant across all platforms. For example, you may want to use the CPU-only builds on Linux too.

In such cases, the first step is to add the relevant PyTorch index to your `pyproject.toml`:

```
=== "CPU-only"

```toml
[[tool.uv.index]]
name = "pytorch-cpu"
url = "https://download.pytorch.org/whl/cpu"
```

```

explicit = true
```

=== "CUDA 11.8"

```toml
[[tool.uv.index]]
name = "pytorch-cu118"
url = "https://download.pytorch.org/whl/cu118"
explicit = true
```

=== "CUDA 12.6"

```toml
[[tool.uv.index]]
name = "pytorch-cu126"
url = "https://download.pytorch.org/whl/cu126"
explicit = true
```

=== "CUDA 12.8"

```toml
[[tool.uv.index]]
name = "pytorch-cu128"
url = "https://download.pytorch.org/whl/cu128"
explicit = true
```

=== "CUDA 13.0"

```toml
[[tool.uv.index]]
name = "pytorch-cu130"
url = "https://download.pytorch.org/whl/cu130"
explicit = true
```

=== "ROCm 7.2"

```toml
[[tool.uv.index]]
name = "pytorch-rocm"
url = "https://download.pytorch.org/whl/rocm7.2"
explicit = true
```

=== "Intel GPUs"

```toml
[[tool.uv.index]]
name = "pytorch-xpu"
url = "https://download.pytorch.org/whl/xpu"
explicit = true
```

```

We recommend the use of `explicit = true` to ensure that the index is *only* used for torch, torchvision, and other PyTorch-related packages, as opposed to generic dependencies like jinja2, which should continue to be sourced from the default index (PyPI).

Next, update the `pyproject.toml` to point `torch` and `torchvision` to the desired index:

=== "CPU-only"

```
```toml
[tool.uv.sources]
torch = [
 { index = "pytorch-cpu" },
]
torchvision = [
 { index = "pytorch-cpu" },
]
```
```

=== "CUDA 11.8"

PyTorch doesn't publish CUDA builds for macOS. As such, we gate on ``sys_platform`` to instruct uv to use the PyTorch index on Linux and Windows, but fall back to PyPI on macOS:

```
```toml
[tool.uv.sources]
torch = [
 { index = "pytorch-cu118", marker = "sys_platform == 'linux' or sys_platform == 'win32'", },
]
torchvision = [
 { index = "pytorch-cu118", marker = "sys_platform == 'linux' or sys_platform == 'win32'", },
]
```
```

=== "CUDA 12.6"

PyTorch doesn't publish CUDA builds for macOS. As such, we gate on ``sys_platform`` to instruct uv to limit the PyTorch index to Linux and Windows, falling back to PyPI on macOS:

```
```toml
[tool.uv.sources]
torch = [
 { index = "pytorch-cu126", marker = "sys_platform == 'linux' or sys_platform == 'win32'", },
]
torchvision = [
 { index = "pytorch-cu126", marker = "sys_platform == 'linux' or sys_platform == 'win32'", },
]
```
```

=== "CUDA 12.8"

PyTorch doesn't publish CUDA builds for macOS. As such, we gate on ``sys_platform`` to instruct uv to limit the PyTorch index to Linux and Windows, falling back to PyPI on macOS:

```
```toml
[tool.uv.sources]
torch = [
```

```

 { index = "pytorch-cu128", marker = "sys_platform == 'linux' or sys_platform ==
'win32'" },
]
torchvision = [
 { index = "pytorch-cu128", marker = "sys_platform == 'linux' or sys_platform ==
'win32'" },
]
...

```

=== "CUDA 13.0"

PyTorch doesn't publish CUDA builds for macOS. As such, we gate on `sys\_platform` to instruct uv to limit the PyTorch index to Linux and Windows, falling back to PyPI on macOS:

```

...toml
[tool.uv.sources]
torch = [
 { index = "pytorch-cu130", marker = "sys_platform == 'linux' or sys_platform ==
'win32'" },
]
torchvision = [
 { index = "pytorch-cu130", marker = "sys_platform == 'linux' or sys_platform ==
'win32'" },
]
...

```

=== "ROCm 7.2"

PyTorch doesn't publish ROCm builds for macOS or Windows. As such, we gate on `sys\_platform` to instruct uv to limit the PyTorch index to Linux, falling back to PyPI on macOS and Windows:

```

...toml
[tool.uv.sources]
torch = [
 { index = "pytorch-rocm", marker = "sys_platform == 'linux'" },
]
torchvision = [
 { index = "pytorch-rocm", marker = "sys_platform == 'linux'" },
]
ROCm support relies on both Triton packages, which should also be installed from
the PyTorch index (and included in `project.dependencies`).
pytorch-triton-rocm = [
 { index = "pytorch-rocm", marker = "sys_platform == 'linux'" },
]
triton-rocm = [
 { index = "pytorch-rocm", marker = "sys_platform == 'linux'" },
]
...

```

=== "Intel GPUs"

PyTorch doesn't publish Intel GPU builds for macOS. As such, we gate on `sys\_platform` to instruct uv to limit the PyTorch index to Linux and Windows, falling back to PyPI on macOS:

```

...toml
[tool.uv.sources]

```

```

torch = [
 { index = "pytorch-xpu", marker = "sys_platform == 'linux' or sys_platform ==
'win32'" },
]
torchvision = [
 { index = "pytorch-xpu", marker = "sys_platform == 'linux' or sys_platform ==
'win32'" },
]
Intel GPU support relies on `triton-xpu`, which should also be installed from the
PyTorch index
(and included in `project.dependencies`).
triton-xpu = [
 { index = "pytorch-xpu", marker = "sys_platform == 'linux' or sys_platform ==
'win32'" },
]
...

```

As a complete example, the following project would use PyTorch's CPU-only builds on all platforms:

```

[project]
name = "project"
version = "0.1.0"
requires-python = ">=3.14.0"
dependencies = [
 "torch>=2.11.0",
 "torchvision>=0.26.0",
]

[tool.uv.sources]
torch = [
 { index = "pytorch-cpu" },
]
torchvision = [
 { index = "pytorch-cpu" },
]

[[tool.uv.index]]
name = "pytorch-cpu"
url = "https://download.pytorch.org/whl/cpu"
explicit = true

```

## Configuring accelerators with environment markers

In some cases, you may want to use CPU-only builds in one environment (e.g., macOS and Windows), and CUDA-enabled builds in another (e.g., Linux).

With `tool.uv.sources`, you can use environment markers to specify the desired index for each platform. For example, the following configuration would use PyTorch's CUDA-enabled builds on Linux, and CPU-only builds on all other platforms (e.g., macOS and Windows):

```

[project]
name = "project"
version = "0.1.0"
requires-python = ">=3.14.0"
dependencies = [
 "torch>=2.11.0",
 "torchvision>=0.26.0",
]

```

```

[tool.uv.sources]
torch = [
 { index = "pytorch-cpu", marker = "sys_platform != 'linux'" },
 { index = "pytorch-cu130", marker = "sys_platform == 'linux'" },
]
torchvision = [
 { index = "pytorch-cpu", marker = "sys_platform != 'linux'" },
 { index = "pytorch-cu130", marker = "sys_platform == 'linux'" },
]

[[tool.uv.index]]
name = "pytorch-cpu"
url = "https://download.pytorch.org/whl/cpu"
explicit = true

[[tool.uv.index]]
name = "pytorch-cu130"
url = "https://download.pytorch.org/whl/cu130"
explicit = true

```

Similarly, the following configuration would use PyTorch's AMD GPU builds on Linux, and CPU-only builds on Windows and macOS (by way of falling back to PyPI):

```

[project]
name = "project"
version = "0.1.0"
requires-python = ">=3.14.0"
dependencies = [
 "torch>=2.11.0",
 "torchvision>=0.26.0",
 "pytorch-triton-rocm>=3.5.1 ; sys_platform == 'linux'",
 "triton-rocm>=3.6.0 ; sys_platform == 'linux'",
]

[tool.uv.sources]
torch = [
 { index = "pytorch-rocm", marker = "sys_platform == 'linux'" },
]
torchvision = [
 { index = "pytorch-rocm", marker = "sys_platform == 'linux'" },
]
pytorch-triton-rocm = [
 { index = "pytorch-rocm", marker = "sys_platform == 'linux'" },
]
triton-rocm = [
 { index = "pytorch-rocm", marker = "sys_platform == 'linux'" },
]

[[tool.uv.index]]
name = "pytorch-rocm"
url = "https://download.pytorch.org/whl/rocm7.2"
explicit = true

```

Or, for Intel GPU builds:

```

[project]
name = "project"

```

```

version = "0.1.0"
requires-python = ">=3.14.0"
dependencies = [
 "torch>=2.11.0",
 "torchvision>=0.26.0",
 "triton-xpu>=3.7.0 ; sys_platform == 'win32' or sys_platform == 'linux'",
]

[tool.uv.sources]
torch = [
 { index = "pytorch-xpu", marker = "sys_platform == 'win32' or sys_platform == 'linux'" },
]
torchvision = [
 { index = "pytorch-xpu", marker = "sys_platform == 'win32' or sys_platform == 'linux'" },
]
triton-xpu = [
 { index = "pytorch-xpu", marker = "sys_platform == 'win32' or sys_platform == 'linux'" },
]

[[tool.uv.index]]
name = "pytorch-xpu"
url = "https://download.pytorch.org/whl/xpu"
explicit = true

```

## Configuring accelerators with optional dependencies

In some cases, you may want to use CPU-only builds in some cases, but CUDA-enabled builds in others, with the choice toggled by a user-provided extra (e.g., `uv sync --extra cpu` vs. `uv sync --extra cu130`).

With `tool.uv.sources`, you can use extra markers to specify the desired index for each enabled extra. For example, the following configuration would use PyTorch's CPU-only for `uv sync --extra cpu` and CUDA-enabled builds for `uv sync --extra cu130`:

```

[project]
name = "project"
version = "0.1.0"
requires-python = ">=3.14.0"
dependencies = []

[project.optional-dependencies]
cpu = [
 "torch>=2.11.0",
 "torchvision>=0.26.0",
]
cu130 = [
 "torch>=2.11.0",
 "torchvision>=0.26.0",
]

[tool.uv]
conflicts = [
 [
 { extra = "cpu" },

```

```

 { extra = "cu130" },
],
]

[tool.uv.sources]
torch = [
 { index = "pytorch-cpu", extra = "cpu" },
 { index = "pytorch-cu130", extra = "cu130" },
]
torchvision = [
 { index = "pytorch-cpu", extra = "cpu" },
 { index = "pytorch-cu130", extra = "cu130" },
]

[[tool.uv.index]]
name = "pytorch-cpu"
url = "https://download.pytorch.org/whl/cpu"
explicit = true

[[tool.uv.index]]
name = "pytorch-cu130"
url = "https://download.pytorch.org/whl/cu130"
explicit = true

```

!!! note

Since GPU-accelerated builds aren't available on macOS, the above configuration will fail to install on macOS when the `cu130` extra is enabled.

## Installing GPU-enabled PyTorch extensions

Many packages in the PyTorch ecosystem include GPU-enabled extensions that are compiled for a specific combination of CUDA and PyTorch versions. Building these packages from source often requires access to the CUDA development toolkit and additional build configuration.

The Astral GPU indexes provide pre-built wheels for packages like `flash-attn`, `deepspeed`, `deep-gemm`, `torch-scatter`, and `vllm`, across a range of Python, CUDA, and PyTorch versions.

To install `flash-attn` from the index for CUDA 12.8, run:

```
$ uv add flash-attn --index astral-cu128=https://wheels.astral.sh/simple/cu128/
```

The command adds `flash-attn` to the project dependencies, configures the Astral GPU index, and pins `flash-attn` to that index.

As with the PyTorch indexes, set `explicit = true` to restrict the Astral GPU index to packages that are explicitly pinned to it:

```

[tool.uv.sources]
flash-attn = { index = "astral-cu128" }

[[tool.uv.index]]
name = "astral-cu128"
url = "https://wheels.astral.sh/simple/cu128/"
explicit = true

```

Each Astral GPU index targets a specific CUDA version, and its wheels are built for specific PyTorch versions. For example, a wheel with a `+cu.12.8.torch.2.11` local version is built for CUDA 12.8 and

PyTorch 2.11. Select the index and wheel that match your Python version, platform, CUDA version, and PyTorch installation.

For the available packages, CUDA versions, and PyTorch versions, see the Astral GPU indexes.

## The uv pip interface

While the above examples are focused on uv's project interface (`uv lock`, `uv sync`, `uv run`, etc.), PyTorch can also be installed via the `uv pip` interface.

PyTorch itself offers a dedicated interface to determine the appropriate pip command to run for a given target configuration. For example, you can install stable, CPU-only PyTorch on Linux with:

```
$ pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu
```

To use the same workflow with uv, replace `pip3` with `uv pip`:

```
$ uv pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu
```

## Automatic backend selection

uv supports automatic selection of the appropriate PyTorch index via the `--torch-backend=auto` command-line argument (or the `UV_TORCH_BACKEND=auto` environment variable), as in:

```
$ # With a command-line argument.
$ uv pip install torch --torch-backend=auto
```

```
$ # With an environment variable.
$ UV_TORCH_BACKEND=auto uv pip install torch
```

When enabled, uv will query for the installed CUDA driver, AMD GPU versions, and Intel GPU presence, then use the most-compatible PyTorch index for all relevant packages (e.g., `torch`, `torchvision`, etc.). If no such GPU is found, uv will fall back to the CPU-only index. uv will continue to respect existing index configuration for any packages outside the PyTorch ecosystem.

You can also select a specific backend (e.g., CUDA 13.0) with `--torch-backend=cu130` (or `UV_TORCH_BACKEND=cu130`):

```
$ # With a command-line argument.
$ uv pip install torch torchvision --torch-backend=cu130
```

```
$ # With an environment variable.
$ UV_TORCH_BACKEND=cu130 uv pip install torch torchvision
```

At present, `--torch-backend` is only available in the `uv pip` interface.